The problem is that the input was read as strings, not numbers. The plus operator for strings concatenates them, so that's what we see in the output. To fix this, use the Integer method to convert the strings to integers:

```
some_file.each do |line|
  v1, v2 = line.split
  print Integer(v1) + Integer(v2), " "
end
```

produces:

```
7 11 15
```

How Numbers Interact

Most of the time, numbers work the way you'd expect. If you perform some operation between two numbers of the same class, the answer will typically be a number of that same class. If the two numbers are different classes, the result will have the class of the more general one. If you mix integers and floats, the result will be a float; if you mix floats and complex numbers, the result will be complex.

```
1 + 2      # => 3
1 + 2.0    # => 3.0
1.0 + 2    # => 3.0
1.0 + 1+2i # => (2.0+2i)
1 + 2/3r   # => (5/3)
1.0 + 2/3r # => 1.6666666666666665
```

The return-type rule still applies when it comes to division. However, this often confuses folks, because division between two integers yields an integer result. If you want integer division to yield a float, you can either convert one side of the division with the to_f method, or multiply one side by 1.0, which is the same thing, or use the fdiv method

```
1.0 / 2     # => 0.5
1 / 2.0     # => 0.5
1 / 2       # => 0
1.to_f / 2  # => 0.5
1 * 1.0 / 2 # => 0.5
1.fdiv(2)   # => 0.5
```

Looping Using Numbers

Integers also support several iterators. We've seen one already: 5.times. Others include upto and downto for iterating up and down between two integers. Class Numeric also provides the more general method step, which is more like a traditional for loop.

```
3.times { print "X " }
1.upto(5) { |i| print i, " " }
99.downto(95) { |i| print i, " " }
50.step(80, 5) { |i| print i, " " }
```

produces:

```
X X X 1 2 3 4 5 99 98 97 96 95 50 55 60 65 70 75 80
```

As with other iterators, if you leave the block off, the call returns an Enumerator.

```
10.downto(7).with_index { |num, index| puts "#{index}: #{num}" }
```